

Übersetzung und Analyse von Programmen

Matthias Janßen
Matrikelnummer: 1871808
Universität Trier

December 15, 2025

Abgabe: Assignment / Sheet

Betreuer: Prof. Dr. Stephan Diehl
Jan Niclas Ruppenthal, M.Sc.

Aufgaben

1 Aufgabe 1

Startzustand: $\rho = \emptyset$

$\text{Nav} = 0$

$\text{Nas} = 0$

$\text{Loc} = 0$

Klasse A

$\text{elab_def}(\text{class A}\{\dots\}) = \text{elab_def } \emptyset 0 0 0$

$\text{elab_def}(B\ x; \text{long myMethod}(\dots) \{\dots\}) \emptyset 0 0 0 =$

$\text{let } (\rho_1, \text{Nav}_1, \text{Nas}_1, \text{Loc}_1) = \text{elab_def}(B\ x; \emptyset 0 0 0)$

$\text{in } \text{elab_def}(\text{long myMethod}(\dots) \{\dots\} \rho_1 \text{Nav}_1 \text{Nas}_1 \text{Loc}_1) \text{ end}$

B x

$\text{elab_def}(B\ x; \rho 0 0 0) = (\rho[(B, 0)/x], 0 + 1, 0, 0)$

$\rho_1 = \{x \mapsto (B, 0)\}$

$\text{Nav}_1 = 1,$

$\text{Nas}_1 = 0,$

$\text{Loc}_1 = 0$

myMethod

$\text{elab_def}(\text{long myMethod}(B\ a, A[]\ b)\{\dots\} \rho_1 1 0 0) =$

$\text{let } (\rho_2, \text{Nav}_2, \text{Nas}_2, \text{Loc}_2) =$

$\text{elab_def } b \quad \rho_1[(B, 1)/a][(A[], 2)/b] 0 0 3$

$\text{in } (\rho_1[(\text{long}, \text{Loc}_2, \rho_2)/\text{myMethod}], 1, 0, 0) \text{ end}$

Parametererstellung

$\rho_1[(B, 1)/a][(A[], 2)/b] = \{x \mapsto (B, 0), a \mapsto (B, 1), b \mapsto (A[], 2)\} = \rho_{\text{params}}$

$\text{elab_def}(\text{int } k = 0) \ \rho_{\text{params}} \ 0 \ 0 \ 3 = \rho_{\text{params}}[(\text{int}, 3)/x], 0, 0, 3 + 1$

$\rho_2 = \rho_{\text{params}} \cup \{k \mapsto (\text{int}, 3)\}$

$\text{nav}_2 = 0$

$\text{nas}_2 = 0$

$\text{loc}_2 = 4$

in: $\rho_1[(\text{long}, \text{Loc}_2, \rho_2)/\text{myMethod}] = \{x \mapsto (B, 0), \text{myMethod} \mapsto (\text{long}, 4, \rho_2)\}$

ρ Klasse A:

$\rho_A = \{x \mapsto (B, 0), \text{myMethod} \mapsto (\text{long}, 4, \rho_2)\}$

$\text{Nav}_A = 1$

$\text{Nas}_A = 0$

$\text{Loc}_A = 0$

Klasse B (extends A)

$\rho = \rho_A$

$\text{Nav} = 1$

$\text{Nas} = 0$

$\text{Loc} = 0$

$\text{elab_def}(\text{class B extends A}\{\dots\}) = \text{elab_def } b \ \rho_A \ 1 \ 0 \ 0$

static long y

$\text{elab_def}(\text{static long } y; \rho_A \ 1 \ 0 \ 0) = (\rho_A[(\text{long}, 0)/y], 1, 0 + 1, 0)$

$\rho_3 = \rho_A \cup \{y \mapsto (\text{long}, 0)\}$

$\text{Nav}_3 = 1, \text{Nas}_3 = 1, \text{Loc}_3 = 0$

myMethod

$\text{elab_def}(\text{long myMethod}(A[] \ a)\{\dots\} \rho_3 \ 1 \ 1 \ 0) =$

$\text{let } (\rho_4, \text{Nav}_4, \text{Nas}_4, \text{Loc}_4) =$

$\text{elab_def } \{b_{\text{body}}\} \ \rho_3[(A[], 1)/a] \ 0 \ 1 \ 2$

$\text{in } (\rho_3[(\text{long}, \text{Loc}_4, \rho_4)/\text{myMethod}], 1, 1, 0) \text{ end}$

Parametererstellung

$\rho_3[(A[], 1)/a] = \{x \mapsto (B, 0), \text{myMethod} \mapsto \dots, y \mapsto (\text{long}, 0), a \mapsto (A[], 1)\} = \rho_{\text{params}}$

Methodenrumpf

$\text{elab_def}(b) \ \rho_{\text{params}} \ 0 \ 1 \ 2$

Im Methodenrumpf wird die Umgebung ρ nicht verändert

$\rho_4 = \{x \mapsto (B, 0), \text{myMethod} \mapsto \dots, y \mapsto (\text{long}, 0), a \mapsto (A[], 1)\}$

$\text{Loc}_4 = 2, \text{Nav}_4 = 0, \text{Nas}_4 = 1$

ρ Klasse B:

$\rho_B = \{x \mapsto (B, 0), y \mapsto (\text{long}, 0), \text{myMethod} \mapsto (\text{long}, 2, \rho_4)\}$

$\text{Nav}_B = 1$

$Nas_B = 1$

$Nas_B = 1$

Bytecode-Übersetzung

Adressumgebung: $\rho_2 = \{x \mapsto (B, 0), a \mapsto (A[], 1), b \mapsto (A[], 2), k \mapsto (int, 3)\}$

int k = 0;

code(0) $\rho_2 = \text{iconst_0}$

istore 3

b[k].x = a;

code(b) $\rho_2 = \text{aload 2}$

code(k) $\rho_2 = \text{iload 3}$

aaload

code(a) $\rho_2 = \text{aload 1}$

putfield

B.y = B.y + 3;

getstatic

code(3) $\rho_2 = \text{bipush 3}$

ladd

putstatic

return B.y;

getstatic

lreturn

B.myMethod(A[] a)

Adressumgebung: $\rho_4 = \{a \mapsto 1\}$

y = y + 1;

getstatic

code(1) $\rho_4 = \text{lconst_1}$

ladd

putstatic

return myMethod(a);

code(a) $\rho_4 = \text{aload 1}$

invokevirtual

lreturn

2 Aufgabe 2

a) Sichtbare Methoden in Klasse C

Berechnung von `vis_meths(C)`

`vis_meths(C)` = $\{m(A, \text{double}), m(B, \text{double}), m(A, \text{int}, \text{int}), m(B, \text{int}), m(C, \text{boolean}), m(A, \text{int})\}$

Erklärung der Methoden:

- $m(A, \text{double})$ – ursprünglich in A definiert, sichtbar in C
- $m(B, \text{double})$ – überschreibt $m(A, \text{double})$ in B, sichtbar in C
- $m(A, \text{int}, \text{int})$ – in A definiert mit zwei int-Parametern, sichtbar in C
- $m(B, \text{int})$ – in B definiert, sichtbar in C
- $m(C, \text{boolean})$ – in C selbst definiert
- $m(A, \text{int})$ – in A definiert mit einem int-Parameter, sichtbar in C

Methoden-Überschreibungen in C:

- Die Methode $m(A, \text{double})$ aus Klasse B wird in Klasse C überschrieben (gemäß Klassendefinition)
- Die Methode $m(A, \text{int}, \text{int})$ aus Klasse A wird ebenfalls in Klasse C überschrieben
- Alle anderen Methoden bleiben sichtbar und unverändert

b) Anwendbare Methoden bei Aufruf $d.m(c, \text{int})$

Gegeben: d ist vom Typ D, Methodenaufruf: $d.m(c, \text{int})$, wobei c vom Typ C ist.

Berechnung von $\text{applicable}(D, (C, \text{int}))$

$$\text{applicable}(D, (C, \text{int})) = \{m(B, \text{double}), m(A, \text{double}), m(B, \text{int}), m(A, \text{int})\}$$

Analyse der Anwendbarkeit: Eine Methode $m(T_1, T_2, \dots)$ ist anwendbar für den Aufruf $m(S_1, S_2, \dots)$, wenn:

$$S_i \sqsubseteq T_i \text{ für alle Parameter } i$$

Hierbei gilt: $D \sqsubseteq C \sqsubseteq B \sqsubseteq A$ (Vererbungshierarchie) und $\text{int} \sqsubseteq \text{double}$ (Java Typ-Kompatibilität).

Überprüfung jeder Methode:

- $m(B, \text{double})$: Da $C \sqsubseteq B$ und $\text{int} \sqsubseteq \text{double}$, ist die Methode **anwendbar**.
- $m(A, \text{double})$: Da $C \sqsubseteq A$ und $\text{int} \sqsubseteq \text{double}$, ist die Methode **anwendbar**.
- $m(B, \text{int})$: Da $C \sqsubseteq B$ und $\text{int} = \text{int}$, ist die Methode **anwendbar**.
- $m(C, \text{boolean})$: Zwar ist $C \sqsubseteq C$, aber $\text{int} \not\sqsubseteq \text{boolean}$, die Methode ist **nicht anwendbar**.
- $m(A, \text{int}, \text{int})$: Der Aufruf hat nur einen int-Parameter, die Methode ist **nicht anwendbar**.
- $m(A, \text{int})$: Da $C \sqsubseteq A$ und $\text{int} = \text{int}$, ist die Methode **anwendbar**.

Mehrdeutigkeit des Aufrufs: Der Methodenaufruf $d.m(c, \text{int})$ ist **nicht eindeutig**, da insgesamt vier anwendbare Methoden existieren:

- $m(B, \text{double})$
- $m(A, \text{double})$
- $m(B, \text{int})$
- $m(A, \text{int})$